FINTRACK

SMART PERSONAL FINANCE MANAGEMENT SYSTEM

MINI PROJECT REPORT

Project Title: FinTrack – Smart Personal Finance Management System
Project Domain: Object-Oriented Programming (OOPS)
Programming Language: C++
Frontend Technologies: HTML, CSS, JavaScript
Backend Technology: C++ with Crow Framework
Database: SQLite
Development Environment: Visual Studio Code

Submitted By: Palak Thakur
Course: BCA
Semester: 3rd
Section: 25BCA3-A
Submitted To: Rashmi
University: Chandigarh University

INDEX

S. No.	Topic
1	Abstract
2	Introduction
3	Problem Statement
4	Objectives of the Project
5	Scope of the Project
6	Existing System
7	Proposed System

8 Technologies Used

9 System Requirements

10 System Architecture

11 Major Modules of FinTrack

12 Object-Oriented Programming Implementation

13 Database Design

14 Working and Data Flow

15 Security and Validation

16 Testing and Expected Results

17 Advantages of the System

18 Limitations

19 Future Enhancements

20 Conclusion

21 References

---

1. ABSTRACT

FinTrack – Smart Personal Finance Management System is a software application designed to help individuals manage their personal financial activities in an organized and efficient manner. In today's world, managing money has become an important part of everyday life. People receive income from different sources and spend money on various activities such as food, transportation, education, shopping, entertainment and other personal requirements. However, many individuals do not maintain proper records of their income and expenses, which makes it difficult to understand their financial condition.

FinTrack aims to solve this problem by providing a centralized platform where users can record, manage and analyze their financial transactions. The application allows users to add income, record expenses, organize transactions into categories, set monthly budgets, monitor savings and generate financial summaries.

The project is developed using a full-stack architecture. The frontend is designed using HTML, CSS and JavaScript to provide an interactive and user-friendly interface. The backend is developed using C++ with the Crow framework, which handles application logic and communication between the frontend and database. SQLite is used to store user information, financial transactions, budgets and savings goals.

An important aspect of this project is the implementation of Object-Oriented Programming concepts. Concepts such as classes, objects, encapsulation, inheritance, abstraction, polymorphism and composition are used to develop a structured and reusable application.

The main purpose of FinTrack is to make personal financial management easier, more transparent and more accessible. It helps users understand their spending habits, control unnecessary expenses and develop better saving practices.

2. INTRODUCTION

Money management is an essential part of modern life. Every individual needs to manage income, expenses and savings to maintain financial stability. Whether a person is a student, employee, business owner or freelancer, keeping track of financial activities helps in making better financial decisions.

In traditional methods, people often maintain their financial records using notebooks, diaries or basic spreadsheets. Although these methods can be useful, they require manual calculations and regular updates. As the number of transactions increases, maintaining accurate records becomes difficult.

With the development of technology, digital financial management applications have become increasingly popular. These applications allow users to maintain their financial records electronically and access important information through a single platform.

FinTrack is designed as a personal finance management application that simplifies this process. It provides users with an organized dashboard where they can view their financial information, including total income, total expenses, available balance and savings.

The application also provides features such as expense categorization, budget management, transaction history and graphical financial reports. These features help users identify where their money is being spent and understand their financial habits.

The project is developed using C++ as its core backend programming language. C++ is selected because it supports Object-Oriented Programming, provides efficient performance and allows developers to create modular and reusable software systems.

The frontend uses HTML, CSS and JavaScript to create a responsive and interactive interface. The backend processes user requests and performs calculations, while SQLite stores the information permanently.

FinTrack is not only designed to solve a practical financial management problem but also demonstrates how OOPS concepts can be applied in a real-world software development project.

3. PROBLEM STATEMENT

Managing personal finances is a common challenge for many individuals. People frequently make multiple transactions throughout the day, but they do not always record these transactions properly. As a result, they may lose track of their spending and find it difficult to calculate their remaining balance.

Some major problems associated with traditional financial management methods are:

1. **Manual record keeping:** Maintaining financial records in notebooks or spreadsheets requires regular manual updates.
2. **Difficulty in tracking expenses:** People often forget small expenses, which can result in inaccurate financial calculations.
3. **Lack of expense categorization:** Without proper categories, it becomes difficult to identify where most of the money is being spent.
4. **Poor budget management:** Many individuals spend more than their planned monthly budget because they do not monitor their expenses regularly.
5. **No centralized financial information:** Income, expenses and savings may be recorded in different places, making financial management complicated.
6. **Time-consuming calculations:** Calculating total income, expenses and savings manually requires additional effort.
7. **Lack of financial analysis:** Traditional methods do not provide easy graphical representations of spending patterns.

Therefore, there is a need for a computerized system that can manage financial transactions, organize expenses, calculate balances and provide useful financial insights.

FinTrack addresses these problems by providing a centralized and user-friendly financial management platform.

4. OBJECTIVES OF THE PROJECT

The primary objective of FinTrack is to develop an efficient personal finance management system that helps users organize their financial activities.

The specific objectives are:

1. To develop a user-friendly application for managing personal finances.
2. To provide secure user registration and login functionality.
3. To allow users to record their income from different sources.
4. To enable users to add and manage daily expenses.
5. To organize transactions into different categories.
6. To calculate total income, total expenses and available balance automatically.
7. To provide a monthly budget management facility.
8. To help users monitor their savings and financial goals.
9. To display financial information through charts and graphical reports.
10. To maintain transaction history for future reference.
11. To implement important Object-Oriented Programming concepts using C++.
12. To connect the frontend, backend and database into one integrated application.

5. SCOPE OF THE PROJECT

The scope of FinTrack covers the management of personal financial information through a digital application.

The application is mainly intended for individuals who want to monitor their income, expenses and savings without using complicated financial software.

The major areas covered by the project include:

5.1 Income Management

Users can add income received from different sources such as salary, pocket money, freelance work, scholarships or other sources.

5.2 Expense Management

Users can record their daily expenses and assign categories such as food, education, transport, shopping, bills and entertainment.

5.3 Budget Management

Users can set monthly spending limits and compare their actual expenses with their planned budgets.

5.4 Savings Tracking

The application can calculate the difference between income and expenses and allow users to monitor their savings goals.

5.5 Financial Reports

The system can generate summaries showing income, expenses, savings and category-wise spending.

5.6 Transaction History

Users can view previously recorded transactions, search records and filter them according to dates and categories.

5.7 User Account Management

Users can register, log in and manage their personal account information.

The initial version focuses on personal finance tracking. It does not directly connect to bank accounts, perform online payments or provide professional investment advice.

6. EXISTING SYSTEM

In the existing traditional system, individuals generally manage their financial information using notebooks, calculators or spreadsheet applications.

These methods allow users to record financial transactions, but they have several limitations.

Disadvantages of the Existing System:

- Financial records need to be maintained manually.
- Calculations may contain human errors.
- Searching for old transactions can be time-consuming.
- There is no automatic expense categorization.
- Budget monitoring requires additional effort.
- Financial reports are not automatically generated.
- Information may become difficult to manage when transaction records increase.

Although spreadsheet applications provide some useful calculation features, they may not offer the complete functionality of a dedicated personal finance management application.

7. PROPOSED SYSTEM

The proposed system is FinTrack – Smart Personal Finance Management System.

It provides a digital solution for managing income, expenses, budgets and savings through a single application.

The system uses a frontend interface to collect user information and display financial data. The backend processes the requests using C++ and communicates with the SQLite database to store and retrieve records.

Major Features of the Proposed System:

1. User registration and authentication.
2. Interactive financial dashboard.
3. Income and expense management.
4. Transaction categorization.
5. Monthly budget creation and monitoring.
6. Savings tracking.
7. Financial charts and reports.
8. Transaction search and filtering.
9. Persistent database storage.
10. Input validation and error handling.

The proposed system reduces manual effort and provides users with a convenient way to understand their financial condition.

Unlike traditional methods, FinTrack can automatically calculate financial summaries whenever a transaction is added, modified or deleted.

8. TECHNOLOGIES USED

The project uses different technologies to develop its frontend, backend and database components.

8.1 C++

C++ is the main programming language used to develop the backend of FinTrack. It is an object-oriented programming language that supports classes, objects, inheritance, polymorphism, abstraction and encapsulation.

It is responsible for processing transactions, calculating financial information, validating inputs and managing application operations.

8.2 HTML

HTML is used to create the basic structure of the frontend. It defines elements such as headings, forms, buttons, tables, navigation bars and dashboard sections.

8.3 CSS

CSS is used to design the appearance of the application. It controls colors, layouts, fonts, spacing, responsiveness and visual elements.

8.4 JavaScript

JavaScript makes the frontend interactive. It handles form submissions, communicates with backend APIs, updates dashboard information and displays transaction data.

8.5 Crow Framework

Crow is a lightweight C++ web framework used to create HTTP servers and REST APIs. It allows the C++ backend to receive requests from the frontend and return responses.

8.6 SQLite

SQLite is a relational database management system used to store application information. It stores user accounts, transactions, budgets and savings goals.

8.7 Chart.js

Chart.js is a JavaScript library used to display graphical financial information. It can generate pie charts, bar charts and line charts for expense analysis and financial summaries.

8.8 Visual Studio Code

Visual Studio Code is the development environment used to write, organize, debug and manage project files.

8.9 Git and GitHub

Git is used for version control, while GitHub is used to host the project repository and maintain its source code.

9. SYSTEM REQUIREMENTS

The system requirements describe the minimum hardware and software needed to develop and operate FinTrack.

9.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 or equivalent
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Display	Standard monitor
Input Devices	Keyboard and mouse
Internet	Required for development resources and online libraries

9.2 Software Requirements

Software	Purpose
Windows 10/11	Operating system
Visual Studio Code	Code editor
C++ Compiler	Compile backend code
CMake	Build configuration
Crow Framework	Backend web server
SQLite	Database management
Web Browser	Run frontend
Git	Version control

10. SYSTEM ARCHITECTURE

FinTrack follows a three-layer architecture consisting of the presentation layer, application layer and database layer.

10.1 Presentation Layer (Frontend)

The presentation layer is the part of the application that users interact with. It is developed using HTML, CSS and JavaScript.

It contains the dashboard, navigation menu, login page, registration page, transaction forms and financial charts.

10.2 Application Layer (Backend)

The application layer is developed using C++ and the Crow framework.

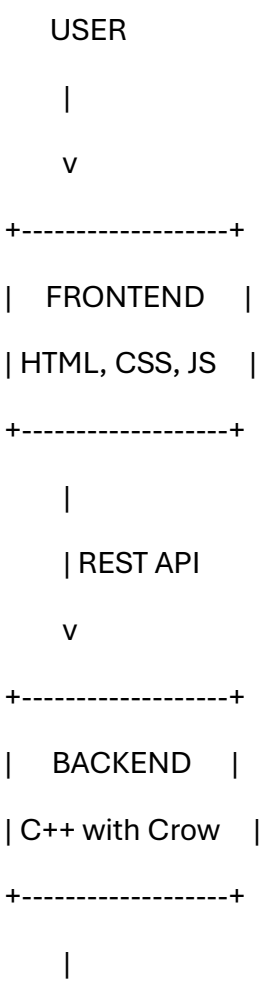
It receives requests from the frontend, validates the information, executes financial calculations and communicates with the database.

10.3 Database Layer

The database layer uses SQLite to store user information, transactions, budgets and savings goals.

It ensures that financial records remain available even after the application is closed.

Architecture Diagram:



Business Logic

OOPS Classes

Validation

Calculations

|

v

+------------------+

| DATABASE |

| SQLite |

+------------------+

|

v

Stored Financial Data

11. MAJOR MODULES OF FINTRACK

The application is divided into different modules to make development and maintenance easier.

11.1 User Registration and Login Module

This module allows new users to create an account and existing users to log in using their credentials.

It verifies user information and ensures that each user can access only their own financial records.

11.2 Dashboard Module

The dashboard provides an overview of the user's financial condition.

It displays:

- Total income.
- Total expenses.
- Available balance.

- Total savings.
- Recent transactions.
- Monthly budget status.
- Financial charts.

11.3 Income Management Module

This module allows users to add, edit and delete income records.

Each income transaction contains details such as amount, source, date and description.

11.4 Expense Management Module

This module allows users to record expenses and organize them into categories.

Examples include food, transportation, shopping, education, entertainment and bills.

11.5 Budget Management Module

Users can create monthly budgets and set spending limits for different categories.

The system compares recorded expenses with the budget and displays the remaining amount.

11.6 Savings Management Module

This module helps users monitor the amount saved and track savings goals.

For example, a user may create a goal to save money for a laptop, mobile phone or emergency fund.

11.7 Transaction History Module

This module maintains a record of all income and expense transactions.

Users can search and filter records according to date, amount or category.

11.8 Reports and Analytics Module

This module presents financial information through charts and summaries.

It helps users understand their spending patterns and identify categories where they spend the most money.

12. OBJECT-ORIENTED PROGRAMMING IMPLEMENTATION

Object-Oriented Programming is the core concept of this project. It allows the application to be divided into different classes that represent real-world entities.

FinTrack uses OOPS principles to create a modular, reusable and maintainable backend.

12.1 Classes and Objects

A class is a blueprint used to create objects. An object is an instance of a class.

In FinTrack, classes represent users, transactions, income, expenses, budgets and the finance manager.

Example:

```
class Budget {

private:

   double monthlyLimit;

   double amountSpent;


public:

   void setBudget(double limit);

   double getRemainingBudget();

};
```

An object of this class can be created to manage a user's monthly budget.

12.2 Encapsulation

Encapsulation is the process of combining data and functions inside a class while restricting direct access to internal data.

In FinTrack, financial information such as transaction amounts and budget limits can be declared private. Public member functions are provided to access and modify this information safely.

This protects data from unauthorized modification within the application.

12.3 Inheritance

Inheritance allows one class to acquire the properties and functions of another class.

FinTrack uses inheritance to represent different types of financial transactions.

For example, Income and Expense classes inherit common transaction properties from the Transaction base class.

```
class Income : public Transaction {

};


class Expense : public Transaction {

};
```

This reduces code duplication and improves reusability.

12.4 Polymorphism

Polymorphism allows the same function interface to perform different operations depending on the object.

In FinTrack, the display() and calculateImpact() functions can be implemented differently by the Income and Expense classes.

For example, income increases the available balance, while expenses decrease it.

12.5 Abstraction

Abstraction hides unnecessary implementation details and exposes only essential functionality.

In FinTrack, an abstract Transaction class defines common operations that must be implemented by its derived classes.

This makes the application easier to extend.

12.6 Composition

Composition represents a relationship in which one class contains objects of other classes.

For example, the FinanceManager class can manage collections of transactions and budgets.

This allows different application components to work together in an organized manner.

12.7 Constructors and Destructors

Constructors initialize objects when they are created. Destructors perform cleanup when objects are destroyed.

FinTrack uses constructors to initialize users, transactions and budgets with appropriate values.

Destructors and resource-management techniques help manage resources safely.

13. DATABASE DESIGN

FinTrack uses SQLite to store and manage financial information.

The database is designed using multiple tables to organize the information and maintain relationships between records.

13.1 Users Table

Stores the account information of registered users.

Field	Description
user_id	Unique user identifier
name	User's name
email	User's email address
password_hash	Secure password hash
created_at	Account creation date

13.2 Transactions Table

Stores income and expense records.

Field	Description
transaction_id	Unique transaction identifier
user_id	Associated user
type	Income or Expense
amount	Transaction amount
category	Transaction category

description	Transaction details
transaction_date	Date of transaction

13.3 Budgets Table

Stores monthly budget information.

Field	Description
budget_id	Unique budget identifier
user_id	Associated user
category	Budget category
monthly_limit	Maximum spending limit
month	Budget month
year	Budget year

13.4 Savings Goals Table

Stores the user's savings targets.

Field	Description
goal_id	Unique goal identifier
user_id	Associated user
goal_name	Name of savings goal
target_amount	Desired savings
current_amount	Current savings
target_date	Expected completion date

Database Relationships

- One user can have multiple transactions.
- One user can create multiple budgets.
- One user can create multiple savings goals.

- Each transaction belongs to one user.

These relationships help organize financial information and prevent unnecessary duplication.

14. WORKING AND DATA FLOW

The working process of FinTrack begins when a user opens the application through a web browser.

First, the user registers or logs in using their credentials. After successful authentication, the dashboard is displayed.

When the user adds a transaction, the frontend collects the required information and sends it to the C++ backend through a REST API request.

The backend validates the received information and checks whether the transaction is valid. If the information is correct, it is stored in the SQLite database.

The backend then retrieves the updated financial information and calculates the total income, total expenses and remaining balance.

Finally, the frontend receives the response and updates the dashboard.

Example of Financial Calculation:

Suppose a user has the following records:

Description	Amount
Monthly income	₹25,000
Freelance income	₹5,000
Food expenses	₹3,000
Transport expenses	₹2,000
Shopping expenses	₹4,000

Total Income = ₹25,000 + ₹5,000 = ₹30,000

Total Expenses = ₹3,000 + ₹2,000 + ₹4,000 = ₹9,000

Available Balance = Total Income − Total Expenses

Available Balance = ₹30,000 − ₹9,000 = ₹21,000

The system automatically calculates these values and displays them on the dashboard.

15. SECURITY AND VALIDATION

Security is an important part of any application that stores personal financial information.

FinTrack includes several security and validation measures.

1. Passwords should be stored as secure hashes instead of plain text.
2. User input should be validated before processing.
3. Database queries should use parameterized statements to reduce SQL injection risks.
4. Users should only be allowed to access their own financial records.
5. Invalid transaction amounts should be rejected.
6. Backend endpoints should verify authentication before accessing private information.
7. Errors should be handled properly without exposing sensitive internal information.

The application should also validate that transaction amounts are positive and that budget limits contain valid numerical values.

16. TESTING AND EXPECTED RESULTS

Testing is performed to verify that the application works correctly and produces accurate results.

The following test cases are planned for the application.

Test Case	Input/Action	Expected Result
User Registration	Enter valid details	Account created
User Login	Correct credentials	Login successful
Invalid Login	Incorrect password	Error message
Add Income	Enter ₹10,000	Income saved
Add Expense	Enter ₹2,000	Expense saved
Invalid Amount	Enter negative amount	Validation error

Budget Creation	Set ₹5,000 limit	Budget saved
View History	Open transaction history	Records displayed
Calculate Balance	Add income and expenses	Correct balance
Delete Transaction	Delete an existing record	Record removed

Expected Results:

After successful implementation, FinTrack should provide a functional financial management platform.

Users should be able to create accounts, add income, record expenses, manage budgets and view financial reports.

The dashboard should update automatically whenever financial transactions are changed.

The database should maintain records permanently, and the backend should process requests correctly.

The application should also demonstrate the practical use of C++ OOPS concepts.

17. ADVANTAGES OF THE SYSTEM

FinTrack provides several advantages over traditional financial management methods.

1. **Easy Financial Management:** Users can manage income and expenses from a single platform.
2. **Time Saving:** Automatic calculations reduce manual effort.
3. **Better Expense Tracking:** Expense categories help users understand their spending habits.
4. **Budget Control:** Monthly budgets help users avoid unnecessary spending.
5. **Improved Savings:** Savings tracking encourages better financial habits.
6. **Centralized Information:** Financial records are stored in one database.
7. **User-Friendly Interface:** The application provides a simple and organized dashboard.
8. **Reusable Code:** OOPS concepts make the backend modular and easier to maintain.

9. **Data Persistence:** SQLite ensures that stored information remains available after closing the application.
10. **Extensibility:** New features can be added in future versions.

18. LIMITATIONS

Although FinTrack provides useful financial management features, the initial version has certain limitations.

1. The application requires users to enter transactions manually.
2. It does not automatically connect to bank accounts.
3. It does not perform real financial transactions or online payments.
4. Its accuracy depends on the information entered by users.
5. Advanced financial forecasting is not included in the initial version.
6. The application does not provide professional investment or tax advice.
7. The initial version is designed mainly for individual users.

These limitations can be addressed through future development.

19. FUTURE ENHANCEMENTS

FinTrack can be improved by adding more advanced features in future versions.

19.1 AI-Based Expense Analysis

Artificial intelligence can be integrated to analyze spending patterns and provide personalized suggestions for reducing unnecessary expenses.

19.2 Bank Account Integration

Secure banking integrations could allow users to import transaction information automatically, subject to supported services and user authorization.

19.3 Mobile Application

A mobile version can be developed for Android and iOS so users can manage their finances anywhere.

19.4 Bill and Payment Reminders

The system can provide reminders for upcoming bills, subscriptions and other financial obligations.

19.5 Advanced Financial Reports

More detailed reports can be generated to compare spending across different months and categories.

19.6 Multi-Currency Support

The application can support multiple currencies for users who manage financial activities in different countries.

19.7 Cloud Synchronization

Cloud storage and synchronization can allow users to access their financial records across multiple devices.

19.8 Export Functionality

Users can download their financial reports in PDF or Excel format for future reference.

20. CONCLUSION

FinTrack – Smart Personal Finance Management System is designed to provide an organized and efficient solution for managing personal financial activities.

The project focuses on solving common problems associated with manual financial record keeping, such as difficulty in tracking expenses, managing budgets and calculating savings.

By providing features such as income management, expense tracking, transaction history, budget management and financial reports, the application helps users maintain better control over their money.

The project combines frontend technologies such as HTML, CSS and JavaScript with a C++ backend developed using the Crow framework. SQLite is used to maintain financial records, while Chart.js provides graphical representations of financial information.

One of the most important aspects of the project is the practical implementation of Object-Oriented Programming concepts. Classes, objects, encapsulation, inheritance, abstraction, polymorphism and composition are used to create a modular and reusable software structure.

Developing this project also provides practical experience in frontend development, backend programming, database management, API communication, software testing and version control.

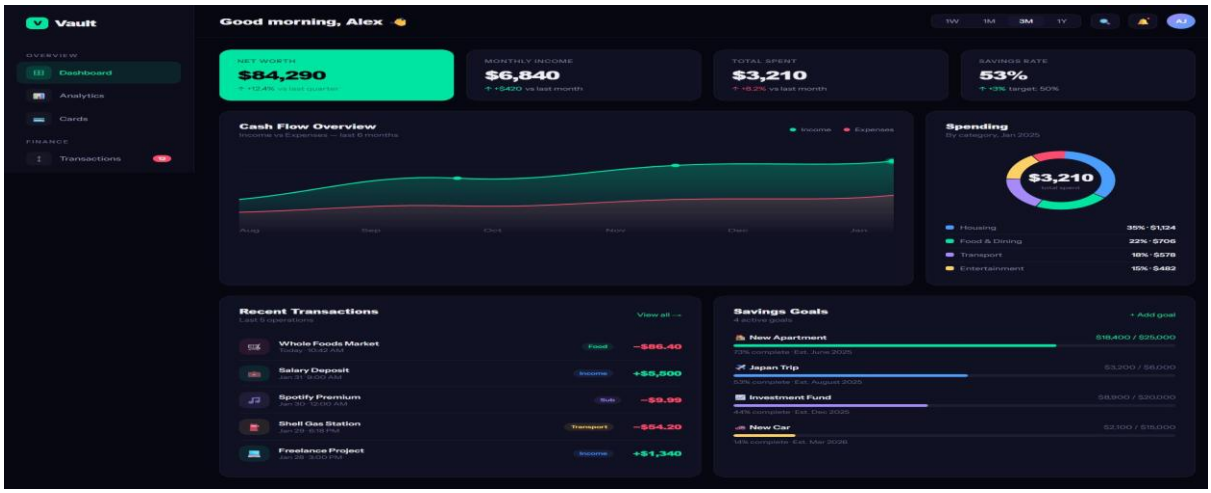
Overall, FinTrack demonstrates how theoretical OOPS concepts can be applied to develop a practical real-world software application. It provides a strong foundation for

future improvements and can be extended into a more advanced personal finance management platform.

21. REFERENCES

1. Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.
2. C++ Reference – https://en.cppreference.com/
3. W3Schools HTML Tutorial – https://www.w3schools.com/html/
4. W3Schools CSS Tutorial – https://www.w3schools.com/css/
5. MDN JavaScript Documentation – https://developer.mozilla.org/en-US/docs/Web/JavaScript
6. Crow C++ Framework Documentation – https://crowcpp.org/
7. SQLite Official Documentation – https://www.sqlite.org/docs.html
8. Chart.js Documentation – https://www.chartjs.org/docs/
9. Git Documentation – https://git-scm.com/doc
10. GitHub Documentation – https://docs.github.com/

Output Images :



CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

